

Goal:

Train a text-conditioned segmentation model so that, given an image and a natural-language prompt, it produces a binary mask for:

1. Visualize data from roboflow:

Crack - seg masks + bbox masks Taping area - bbox masks Have annotation information in coco format.

2. Using SAM model (Segment Anything Model) to generate seg masks for taping area given bbox coordinates.

Saved masks as coco format

```
import torch
from segment_anything import sam_model_registry, SamPredictor
import numpy as np
import cv2
import os

device = "cuda" if torch.cuda.is_available() else "cpu"

sam = sam_model_registry["vit_l"](checkpoint="sam_vit_l_0b3195.pth")
sam.to(device)
predictor = SamPredictor(sam)
```

3. Data Augmentation done in Roboflow, while downloading dataset.

Using 70/15/15 Split for both datasets.

4. Loaded CLIPSeg model:

https://huggingface.co/docs/transformers/en/model_doc/clipseg

```
processor = CLIPSegProcessor.from_pretrained("CIDAS/clipseg-rd64-refined")
model =
CLIPSegForImageSegmentation.from_pretrained("CIDAS/clipseg-rd64-refined").to(device)
```

Model Size: Total params: 116M

5. Training Strategy & Loss Functions

- Implemented a multi-loss objective combining
 - BCEWithLogitsLoss (pixel-wise classification)
 - Dice Loss (shape-aware overlap)
- Loss = BCE + Dice
- Resized model logits to match the mask resolution
- Metrics computed per batch: IoU, Dice coefficient
- Added reproducibility: manual seed set for PyTorch & NumPy

6. Model Training Setup

- Base model: CLIPSeg (rd64-refined) loaded using HuggingFace Transformers
- Optimizer: AdamW, LR = 1e-4
- Batch size: 4
- Training performed on combined dataset (taping + cracks)
- Validation performed separately on both tasks
- Runtime metrics logged every epoch (loss, IoU, Dice, inference time)

```
2026-01-23 16:55:21.090050: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to regist
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1769187321.130073 5442 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to reg
E0000 00:00:1769187321.144279 5442 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to r
W0000 00:00:1769187321.176906 5442 computation_placer.cc:177] computation placer already registered. Ple
W0000 00:00:1769187321.176957 5442 computation_placer.cc:177] computation placer already registered. Ple
W0000 00:00:1769187321.176967 5442 computation_placer.cc:177] computation placer already registered. Ple
W0000 00:00:1769187321.176975 5442 computation_placer.cc:177] computation placer already registered. Ple
2026-01-23 16:55:21.184140: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriat
Using a slow image processor as `use_fast` is unset and a slow processor was saved with this model. `use_fa

===== Epoch 1/10 =====
Training: 0%| 0/1084 [00:00<?, ?it/s]/usr/lo
    return self.preprocess(images, **kwargs)
It looks like you are trying to rescale already rescaled images. If the input images have pixel values betw
Training: 100%| 1084/1084 [05:04<00:00, 3.55it/s]
Validation: 100%| 232/232 [00:51<00:00, 4.51it/s]
Train Loss: 0.7691 | Val Loss: 0.7035
mIoU: 0.3358 | Dice: 0.4744 | Avg Inf: 0.0263s
Saved: clipseg_epoch_1.pth (Size: 603.18 MB)

===== Epoch 2/10 =====
Training: 100%| 1084/1084 [03:59<00:00, 4.52it/s]
Validation: 100%| 232/232 [00:48<00:00, 4.77it/s]
Train Loss: 0.6745 | Val Loss: 0.6723
mIoU: 0.3523 | Dice: 0.4922 | Avg Inf: 0.0233s
Saved: clipseg_epoch_2.pth (Size: 603.18 MB)

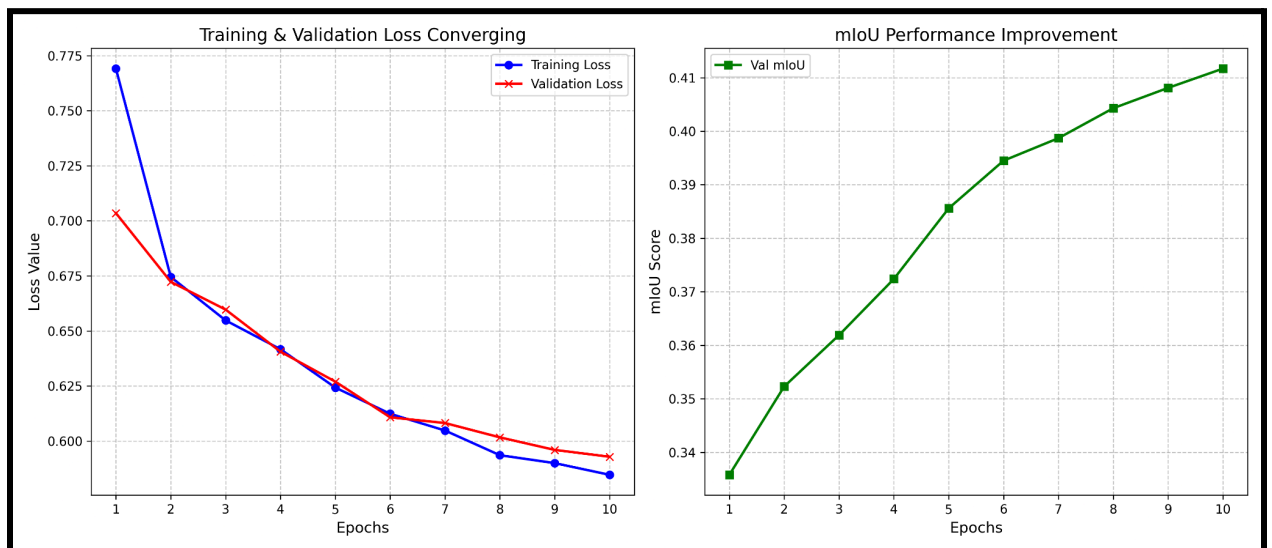
===== Epoch 3/10 =====
Training: 100%| 1084/1084 [04:18<00:00, 4.20it/s]
Validation: 100%| 232/232 [00:50<00:00, 4.62it/s]
Train Loss: 0.6548 | Val Loss: 0.6597
mIoU: 0.3619 | Dice: 0.5024 | Avg Inf: 0.0249s
Saved: clipseg_epoch_3.pth (Size: 603.18 MB)
```

7. Dataset Pipeline

- Custom PyTorch **Dataset** class
- Performs:
 - Uniform image resize to 352×352
 - Text prompt injection per sample (e.g., "**segment_crack**")
 - Mask normalization from **{0,255}** to **{0,1}**
 - Automatic matching of image $\rightleftharpoons$ prompt-specific mask
- Uses two independent transforms:
 - Image: **Resize + ToTensor**
 - Mask: **Resize (NEAREST)** to preserve edges

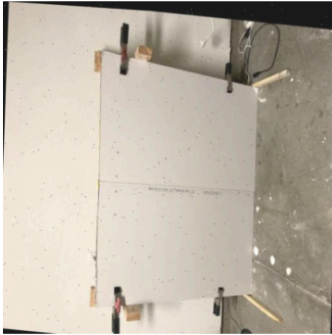
8. Training & Validation Loop

- Per epoch: forward pass $\rightarrow$ logits resized $\rightarrow$ BCE + Dice loss $\rightarrow$ optimizer step
- Validation runs in no-grad mode with same preprocessing
- Computes: val loss, mIoU, mDice, avg inference time
- Model checkpoint saved after every epoch

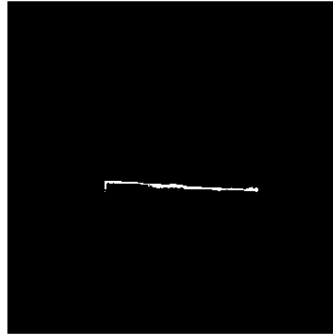


Results while training

Original (segment_taping_area)



Ground Truth



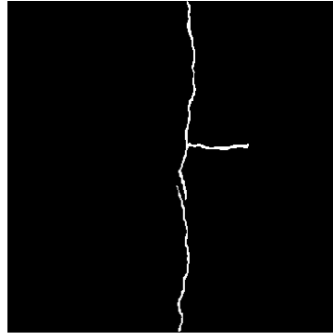
Prediction



Original (segment_crack)



Ground Truth



Prediction



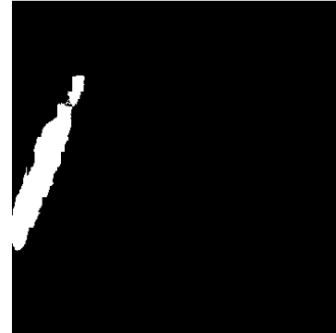
Original (segment_taping_area)



Ground Truth



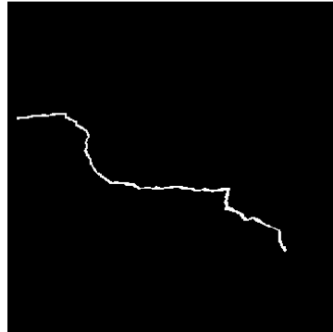
Prediction



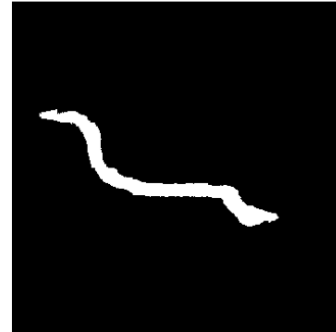
Original (segment_crack)



Ground Truth



Prediction



Failure Notes

- Faced tensor size mismatch between logits and masks, resolved via output resizing.
- HuggingFace processor caused rescaling/normalization conflicts
- .Ran into library version incompatibilities, fixed after aligning PyTorch-Transformers versions.

9. Last Epoch Stats

Train Loss: 0.5847 | Val Loss: 0.5929

mIoU: 0.4117 | Dice: 0.5664 | Avg Inf: 0.0231s

Saved: clipseg_epoch_10.pth (Size: 603.18 MB)

10. Model Link

https://huggingface.co/RohanMulay1/CLIPseg_wall_cracks/blob/main/clipseg_epoch_10.pth